Operators & Expressions

✅ What Are Operators in Java?

Operators in Java are *special symbols or keywords used to perform operations on variables and values.* For example, + is an operator for addition, == for comparison, and so on.

🔹 Types of Operators in Java

Java has **7 main categories** of operators:

---

1. Arithmetic Operators

Used to perform basic mathematical operations.

Operator	Description	Example (int a = 10, b = 5)	Result
+	Addition	a + b	15
-	Subtraction	a - b	5
*	Multiplication	a * b	50
/	Division	a / b	2
%	Modulus (remainder)	a % b	0

✅ **Example:**

```
int a = 10, b = 5;
System.out.println("Addition: " + (a + b));
System.out.println("Subtraction: " + (a - b));
System.out.println("Multiplication: " + (a * b));
System.out.println("Division: " + (a / b));
System.out.println("Modulus: " + (a % b));
```

---

2. Relational / Comparison Operators

Used to compare two values.

Operator	Meaning	Example (a = 10, b = 5)	Result
==	Equal to	a == b	false
!=	Not equal to	a != b	true
>	Greater than	a > b	true
<	Less than	a < b	false
>=	Greater or equal	a >= b	true
<=	Less or equal	a <= b	false

✅ **Example:**

```
int a = 10, b = 5;
System.out.println(a == b); // false
```

```
System.out.println(a != b); // true
System.out.println(a > b);  // true
```

---

3. Logical Operators

Used with boolean values (true/false).

Operator	Description	Example	Result
&&	Logical AND	true && false	false
||		`	Logical OR
!	Logical NOT	!true	false

✅ **Example:**

```
boolean x = true, y = false;
System.out.println(x && y); // false
System.out.println(x || y); // true
System.out.println(!x);     // false
```

---

4. Assignment Operators

Assign values to variables.

Operator	Description	Example	Meaning
=	Assign	a = 5	Assign 5 to a
+=	Add & assign	a += 2	a = a + 2
-=	Subtract	a -= 2	a = a - 2
*=	Multiply	a *= 2	a = a * 2
/=	Divide	a /= 2	a = a / 2

✅ **Example:**

```
int a = 10;
a += 5;    // a = 15   This is equivalent to: a = a + 5
a *= 2;    // a = 30   This is equivalent to: a = a * 2

System.out.println("Final a: " + a);
```

---

5. Unary Operators

Work with only one operand.

Operator	Description	Example	Result
+	Unary plus	+a	10
-	Unary minus	-a	-10
++	Increment	a++	10 → 11
--	Decrement	a--	10 → 9
!	NOT	!true	false

✅ **Example:**

```
int a = 5;
System.out.println(a++); // prints 5, then a becomes 6
System.out.println(++a); // a becomes 7, then prints 7
System.out.println(--a); // a becomes 6, then prints 6
```

---

6. Bitwise Operators

Used for binary/bit-level operations.

Operator	Description	Example	Result (in bits)
&	AND	a & b	0000 0100 (4)
`	`	OR	`a
^	XOR	a ^ b	0000 1010 (10)
~	Complement	~a	Inverts bits
<<	Left shift	a << 1	Multiplies by 2
>>	Right shift	a >> 1	Divides by 2

✅ **Example:**

```
int a = 12, b = 5;
System.out.println(a & b);  // 4
System.out.println(a | b);  // 13
System.out.println(a ^ b);  // 9
System.out.println(~a);     // -13 (bitwise NOT)
System.out.println(a << 1); // 24
System.out.println(a >> 1); // 6
```

---

7. Ternary Operator

Short form of if-else.

Syntax: condition ? trueValue : falseValue

✅ **Example:**

```
int a = 10, b = 20;
int max = (a > b) ? a : b;
System.out.println("Max: " + max); // 20
```

✅ Rules to Remember

1. **Operator precedence** matters: * and / are evaluated before + and -.
2. **Associativity**: Most operators are **left to right**, except = and ternary (? :), which are **right to left**.
3. **Use parentheses** () to control the order of evaluation.
4. **Short-circuit**: && and || stop evaluating if the result is already determined.

Mini Code with All Types

```
package com.Section05_OperatorAndExpression;

public class Exp01_OperatorDemo {

    public static void main(String[] args) {
        int a = 10, b = 3;              // Declare two integer variables a and b
        boolean x = true, y = false;    // Declare two boolean variables x and y

        // Arithmetic Operators
        System.out.println("Arithmetic Operators");
        System.out.println("Addition (a + b): " + (a + b));          // Adds a and b, prints result
        System.out.println("Subtraction (a - b): " + (a - b));       // Subtracts b from a, prints result
        System.out.println("Multiplication (a * b): " + (a * b));    // Multiplies a and b, prints result
        System.out.println("Division (a / b): " + (a / b));          // Divides a by b (integer division), prints result
        System.out.println("Modulus (a % b): " + (a % b));           // Finds remainder when a is divided by b, prints
result

        // Relational Operators
        System.out.println("-----------------------------------------------------");
        System.out.println("Relational Operators");
        System.out.println("a > b: " + (a > b));                     // Checks if a is greater than b, prints boolean result
        System.out.println("a < b: " + (a < b));                     // Checks if a is less than b, prints boolean result
        System.out.println("a == b: " + (a == b));                   // Checks if a equals b, prints boolean result
        System.out.println("a != b: " + (a != b));                   // Checks if a is not equal to b, prints boolean result

        // Logical Operators
        System.out.println("-----------------------------------------------------");
        System.out.println("Logical Operators");
        System.out.println("x && y: " + (x && y));                   // Logical AND of x and y, prints boolean result
        System.out.println("x || y: " + (x || y));                   // Logical OR of x and y, prints boolean result
        System.out.println("!x: " + (!x));                           // Logical NOT of x, prints boolean result

        // Bitwise Operators
        System.out.println("-----------------------------------------------------");
        System.out.println("Bitwise Operators");
        System.out.println("a & b (Bitwise AND): " + (a & b));       // Bitwise AND of a and b, prints integer result
        System.out.println("a | b (Bitwise OR): " + (a | b));        // Bitwise OR of a and b, prints integer result
```

```
        System.out.println("a ^ b (Bitwise XOR): " + (a ^ b));        // Bitwise XOR of a and b, prints integer result
        System.out.println("~a (Bitwise NOT): " + (~a));              // Bitwise NOT of a, prints integer result
        System.out.println("a << 1 (Left Shift): " + (a << 1));       // Left shifts a by 1 bit (multiplies by 2), prints result
        System.out.println("a >> 1 (Right Shift): " + (a >> 1));      // Right shifts a by 1 bit (divides by 2), prints result
        System.out.println("a >>> 1 (Unsigned Right Shift): " + (a >>> 1)); // Unsigned right shift a by 1 bit, prints
result

        // Assignment Operators
        System.out.println("-------------------------------------------------------");
        System.out.println("Assignment Operators");
        a += 5;                                          // Adds 5 to a (a = a + 5)
        System.out.println("a after a += 5: " + a);               // Prints updated a
        a -= 2;                                         // Subtracts 2 from a (a = a - 2)
        System.out.println("a after a -= 2: " + a);               // Prints updated a
        a *= 2;                                          // Multiplies a by 2 (a = a * 2)
        System.out.println("a after a *= 2: " + a);               // Prints updated a
        a /= 3;                                         // Divides a by 3 (a = a / 3)
        System.out.println("a after a /= 3: " + a);               // Prints updated a
        a %= 4;                                           // Assigns remainder of a/4 to a (a = a % 4)
        System.out.println("a after a %= 4: " + a);                // Prints updated a

        // Unary Operators
        System.out.println("-------------------------------------------------------");
        System.out.println("Unary Operators");
        System.out.println("++a (Pre-increment): " + (++a));         // Increments a by 1, then prints
        System.out.println("a++ (Post-increment): " + (a++));         // Prints a, then increments a by 1
        System.out.println("a after a++: " + a);                  // Prints updated a after post-increment
        System.out.println("--a (Pre-decrement): " + (--a));         // Decrements a by 1, then prints
        System.out.println("a-- (Post-decrement): " + (a--));         // Prints a, then decrements a by 1
        System.out.println("Final a value: " + a);                // Prints final value of a

        // Ternary Operator
        System.out.println("-------------------------------------------------------");
        System.out.println("Ternary Operator");
        int max = (a > b) ? a : b;                       // Assigns max to a if a > b, else b
        System.out.println("Max of a and b: " + max);              // Prints the max value
    }
}
```

```
Arithmetic Operators
Addition (a + b): 13
Subtraction (a - b): 7
Multiplication (a * b): 30
Division (a / b): 3
Modulus (a % b): 1
-------------------------------------------------------
Relational Operators
a > b: true
a < b: false
a == b: false
a != b: true
-------------------------------------------------------
Logical Operators
x && y: false
x || y: true
!x: false
-------------------------------------------------------
Bitwise Operators
a & b (Bitwise AND): 2
a | b (Bitwise OR): 11
a ^ b (Bitwise XOR): 9
~a (Bitwise NOT): -11
```

a << 1 (Left Shift): 20
a >> 1 (Right Shift): 5
a >>> 1 (Unsigned Right Shift): 5

Assignment Operators
a after a += 5: 15
a after a -= 2: 13
a after a *= 2: 26
a after a /= 3: 8
a after a %= 4: 0

Unary Operators
++a (Pre-increment): 1
a++ (Post-increment): 1
a after a++: 2
--a (Pre-decrement): 1
a-- (Post-decrement): 1
Final a value: 0

Ternary Operator
Max of a and b: 3

Problem Statement: Student Grading System

[Note:This Question is base on the OOPs concepts]

Objective:

Design and implement a **Java console-based application** that allows a user to input a student's name and their marks in three subjects. The program should then calculate the total marks, average percentage, determine the grade, and print a result summary along with pass/fail status.

Requirements:

1. **Input**:
   - Student Name (String)
   - Marks in:
     - Math (int)
     - Science (int)
     - English (int)
2. **Process**:
   - Calculate total marks.
   - Calculate average percentage.
   - Determine **grade** using this scale:
     - 90+ → Grade A
     - 75–89 → Grade B
     - 60–74 → Grade C
     - 40–59 → Grade D
     - <40 → Grade F
   - A student **passes** if:
     - Average is ≥ 40
     - All individual subjects have marks ≥ 35
3. **Output**:
   - Student name
   - Total marks
   - Average
   - Grade
   - Pass/Fail status

Constraints:

- All marks are out of 100.
- Use methods where possible (e.g., for grade calculation).
- Use operators efficiently (+, /, >=, &&, ternary ? :, etc.)

Bonus:

- Add support for **multiple students** (using loop or arrays).
- Allow user to retry or exit after each student.

Great! Let's now solve the **"Student Grading System"** problem using **object-oriented programming (OOP)** with proper structure, **methods**, **class design**, and **Java operators**.

✅ **SOLUTION: Object-Oriented Java Program**

🔧 **Key Concepts Used:**

- **Class & Object** (Encapsulation)
- **Constructor**
- **Instance variables**
- **Methods** (for calculation and result)
- **Control statements** (if-else)
- **Loops**
- **Operators** (arithmetic, relational, logical, assignment, ternary)

📌 **Step-by-Step Plan**

1. Create a Student class with:
    - Fields: name, math, science, english, total, average, grade
    - Methods:
        - calculateTotalAndAverage()
        - calculateGrade()
        - isPass()
        - displayResult()

2. In the main() method:
    - Take input from the user
    - Create a Student object
    - Call the necessary methods
    - Display result

💻 **Full Working Code**

```
import java.util.Scanner;

class Student {
    // Fields (Attributes)
    private String name;
    private int math;
    private int science;
    private int english;
    private int total;
    private double average;
    private char grade;

    // Constructor
    public Student(String name, int math, int science, int english) {
        this.name = name;
        this.math = math;
        this.science = science;
        this.english = english;
```

```
    }

    // Method to calculate total and average
    public void calculateTotalAndAverage() {
        total = math + science + english; // Arithmetic operator
        average = total / 3.0;            // Division operator
    }

    // Method to determine grade
    public void calculateGrade() {
        if (average >= 90)
            grade = 'A';
        else if (average >= 75)
            grade = 'B';
        else if (average >= 60)
            grade = 'C';
        else if (average >= 40)
            grade = 'D';
        else
            grade = 'F';
    }

    // Method to determine pass/fail
    public boolean isPass() {
        return (average >= 40 && math >= 35 && science >= 35 && english >= 35); // Logical + relational
    }

    // Method to display student result
    public void displayResult() {
        System.out.println("\n===== Student Report =====");
        System.out.println("Name    : " + name);
        System.out.println("Math    : " + math);
        System.out.println("Science : " + science);
        System.out.println("English : " + english);
        System.out.println("Total   : " + total);
        System.out.println("Average : " + average);
        System.out.println("Grade   : " + grade);

        String result = isPass() ? "PASS" : "FAIL"; // Ternary operator
        System.out.println("Status  : " + result);
    }
}

public class StudentGradingSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input section
        System.out.print("Enter Student Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Math marks: ");
        int math = sc.nextInt();

        System.out.print("Enter Science marks: ");
        int science = sc.nextInt();

        System.out.print("Enter English marks: ");
        int english = sc.nextInt();

        // Object creation and method calling
        Student student = new Student(name, math, science, english);

        student.calculateTotalAndAverage();
```

```
        student.calculateGrade();
        student.displayResult();

        sc.close();
    }
}
```

Sample Output:

```
Enter Student Name: Alice
Enter Math marks: 85
Enter Science marks: 78
Enter English marks: 90

===== Student Report =====
Name    : Alice
Math    : 85
Science : 78
English : 90
Total   : 253
Average : 84.33333333333333
Grade   : B
Status  : PASS
```

What You Practiced:

Concept	Used In
Class/Object	Student class and object in main()
Constructor	Initializing student data
Encapsulation	Keeping fields private
Arithmetic Operators	Total and average
Relational Operators	>= for grade and pass check
Logical Operators	&& for pass criteria
Ternary Operator	Pass/Fail result
Method Reusability	Clean modular code
Scanner (User Input)	Read input from console

[Note : Create a new project for better understanding and practices ..]

Would you like to extend this program for **multiple students** using loops or arrays?

Great! Let's now extend the **Student Grading System** to support **multiple students** using:

- Loops (for)
- Array of Student objects
- Input/output for each student
- Reusability via OOP principles

✅ Updated Problem Statement

Modify the program to allow the user to input and display **details for multiple students** (say 3). Use a loop and store student objects in an array.

🧠 Full Working Code: Multiple Students (Array + Loop + OOP)

```
import java.util.Scanner;

class Student {
    private String name;
    private int math, science, english;
    private int total;
    private double average;
    private char grade;

    public Student(String name, int math, int science, int english) {
        this.name = name;
        this.math = math;
        this.science = science;
        this.english = english;
    }

    public void calculateTotalAndAverage() {
        total = math + science + english;
        average = total / 3.0;
    }

    public void calculateGrade() {
        if (average >= 90)
            grade = 'A';
        else if (average >= 75)
            grade = 'B';
        else if (average >= 60)
            grade = 'C';
        else if (average >= 40)
            grade = 'D';
        else
            grade = 'F';
    }
```

```
    public boolean isPass() {
        return (average >= 40 && math >= 35 && science >= 35 && english >= 35);
    }

    public void displayResult() {
        System.out.println("\n===== Student Report =====");
        System.out.println("Name    : " + name);
        System.out.println("Math    : " + math);
        System.out.println("Science : " + science);
        System.out.println("English : " + english);
        System.out.println("Total   : " + total);
        System.out.println("Average : " + average);
        System.out.println("Grade   : " + grade);
        System.out.println("Status  : " + (isPass() ? "PASS" : "FAIL"));
    }
}

public class StudentGradingSystemMultiple {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of students: ");
        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        Student[] students = new Student[n];

        // Input Loop
        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter details for Student " + (i + 1) + ":");

            System.out.print("Name: ");
            String name = sc.nextLine();

            System.out.print("Math Marks: ");
            int math = sc.nextInt();

            System.out.print("Science Marks: ");
            int science = sc.nextInt();

            System.out.print("English Marks: ");
            int english = sc.nextInt();
            sc.nextLine(); // consume newline

            students[i] = new Student(name, math, science, english);
            students[i].calculateTotalAndAverage();
            students[i].calculateGrade();
        }

        // Output Loop
        System.out.println("\n======= Final Results =======");
        for (Student student : students) {
            student.displayResult();
        }

        sc.close();
    }
}
```

Sample Output:

```
Enter number of students: 2
```

```
Enter details for Student 1:
Name: Alice
Math Marks: 80
Science Marks: 90
English Marks: 85

Enter details for Student 2:
Name: Bob
Math Marks: 40
Science Marks: 35
English Marks: 50

======= Final Results =======

===== Student Report =====
Name    : Alice
Math    : 80
Science : 90
English : 85
Total   : 255
Average : 85.0
Grade   : B
Status  : PASS

===== Student Report =====
Name    : Bob
Math    : 40
Science : 35
English : 50
Total   : 125
Average : 41.666666666666664
Grade   : D
Status  : PASS
```

🔁 What You Practiced:

Feature	Description
OOP Design	Student class with methods
Looping	for loop for multiple students
Array of Objects	Student[] students = new Student[n];
Input Handling	Scanner with nextLine() and nextInt()
Code Reuse	Clean, modular, DRY (Don't Repeat Yourself)

Would you like to further enhance this by:

- Saving to file?
- Sorting students by average?
- Adding subject names dynamically?

Let me know!

✅ Valid Data Types for ++ and -- Operators

You can use these operators with **numeric and character data types** that support arithmetic:

Data Type	Description	Example
byte	Small integer (8-bit)	byte b = 10; b++;
short	Small integer (16-bit)	short s = 20; s--;
int	Integer (32-bit)	int i = 5; ++i;
long	Large integer (64-bit)	long l = 100L; l++;
float	Decimal (32-bit)	float f = 1.5f; f++;
double	Decimal (64-bit)	double d = 3.14; d--;
char	Character (16-bit Unicode)	char c = 'A'; c++; → becomes 'B'

❌ Invalid Data Types for ++ and -- Operators

You **cannot** use them with:

Data Type	Reason	Example
boolean	Booleans are not numeric (no +1 or -1)	boolean b = true; b++; // Error
String	Strings are objects, not numbers	String s = "abc"; s++; // Error
Objects	e.g., Scanner, Integer, ArrayList	list++; // Error
final vars	Final vars are constants, can't be changed	final int x = 5; x++; // Error

---

🔎 Example: Valid Use

```
public class ValidIncrement {
    public static void main(String[] args) {
        byte b = 10;
        short s = 100;
        int i = 5;
        long l = 1000L;
        float f = 1.5f;
        double d = 3.14;
        char c = 'A';

        b++;
        s--;
        i++;
        l--;
        f++;
```

```
        d--;
        c++;

        System.out.println("char after increment: " + c); // prints B
    }
}
```

Summary

Operator	Allowed On	Not Allowed On
++ / --	byte, short, int, long, float, double, char	boolean, String, objects, final

---

✅ What Are Bitwise Operators in Java?

Great! Let's explore **Bitwise Operators in Java** — a powerful but often confusing topic. Here's a complete breakdown with rules, examples, and how they work.

✅ What Are Bitwise Operators in Java?

Bitwise operators in Java (and most programming languages) perform operations on individual bits of integer values (byte, short, int, long, and char). They work directly on the binary representation of numbers.

Bitwise operators perform operations **bit-by-bit on integers** (not floats/doubles). They treat numbers as sequences of **binary digits (bits)** — 0s and 1s.

List of Bitwise Operators in Java

Operator	Symbol	Description	Example (binary)
AND	&	1 only if both bits are 1	5 & 3 → 1
OR	`	`	1 if at least one bit is 1
XOR	^	1 if bits are different	5 ^ 3 → 6
NOT	~	Inverts each bit	~5 → -6
Left Shift	<<	Shifts bits to the left	5 << 1 → 10
Right Shift	>>	Shifts bits to the right (signed)	5 >> 1 → 2
Unsigned Right Shift	>>>	Shifts bits to the right, fills 0 from left	-5 >>> 1

Java Bitwise Operators

1. Bitwise AND (&)

- Performs a logical AND on each pair of corresponding bits.
- Result bit is 1 **only if both bits are 1**.

Example:

```
int a = 5;    // 0101 (binary)
```

```
int b = 3;    // 0011 (binary)
int result = a & b;  // 0001 (1 in decimal)
System.out.println(result); // Output: 1
```

---

2. Bitwise OR (|)

- Performs a logical OR on each pair of corresponding bits.
- Result bit is 1 **if at least one of the bits is 1**.

Example:

```
int a = 5;    // 0101
int b = 3;    // 0011
int result = a | b;  // 0111 (7 in decimal)
System.out.println(result); // Output: 7
```

---

3. Bitwise XOR (^)

- Performs an exclusive OR (XOR).
- Result bit is 1 **if the bits are different**, 0 if they are the same.

Example:

```
int a = 5;    // 0101
int b = 3;    // 0011
int result = a ^ b;  // 0110 (6 in decimal)
System.out.println(result); // Output: 6
```

---

4. Bitwise NOT (~)

- Unary operator that flips all bits.
- 1 becomes 0 and 0 becomes 1.

Example:

```
int a = 5;    // 0000 0101
int result = ~a;  // 1111 1010 (two's complement: -6)
System.out.println(result); // Output: -6
```

---

5. Left Shift (<<)

- Shifts bits to the left by a specified number of positions.
- Rightmost bits are filled with 0.
- Equivalent to multiplying by 2^n.

Example:

```
int a = 5;    // 0000 0101
int result = a << 1;  // 0000 1010 (10 in decimal)
System.out.println(result); // Output: 10
```

---

6. Right Shift (>>) (Signed)

- Shifts bits to the right.
- Fills leftmost bits with the **sign bit** (preserves sign).
- Equivalent to dividing by 2^n.

Example:

```
int a = 8;    // 0000 1000
int result = a >> 1;  // 0000 0100 (4 in decimal)
System.out.println(result); // Output: 4
```

---

7. Unsigned Right Shift (>>>)

- Shifts bits to the right.
- Fills leftmost bits with 0 (ignores sign).
- Does **not** preserve the sign bit.

Example:

```
int a = -8;   // 1111 1000 (32-bit two's complement)
int result = a >>> 1;  // 0111 1100... (2147483644 in decimal)
System.out.println(result); // Output: 2147483644
```

Common Use Cases of Bitwise Operators

- **Efficient Arithmetic:** x << 1 is faster than x * 2.
- **Flag Manipulation:** Compactly manage multiple boolean flags.
- **Cryptography & Hashing:** XOR used in encryption.
- **Low-level Hardware Control:** Useful in embedded systems and device drivers.

Bitwise Operator Examples

```
public class BitwiseDemo {
    public static void main(String[] args) {
        int a = 5;  // binary: 0101
        int b = 3;  // binary: 0011

        System.out.println("a & b = " + (a & b)); // 1
        System.out.println("a | b = " + (a | b)); // 7
        System.out.println("a ^ b = " + (a ^ b)); // 6
        System.out.println("~a = " + (~a));       // -6 (2's complement)
        System.out.println("a << 1 = " + (a << 1)); // 10 (0101 -> 1010)
        System.out.println("a >> 1 = " + (a >> 1)); // 2  (0101 -> 0010)
    }
}
```

Binary Calculation Example

For a = 5 and b = 3:

Value	Binary
5	0101
3	0011

a & b → 0101 & 0011 → 0001 → 1

a | b → 0101 | 0011 → 0111 → 7

a ^ b → 0101 ^ 0011 → 0110 → 6

🔍 How Left & Right Shift Work

- a << 1 means **shift all bits left by 1** (multiplies by 2).
- a >> 1 means **shift all bits right by 1** (divides by 2 for positive numbers).
- >>> does the same as >>, but always fills 0 from left (good for unsigned numbers).

❌ Bitwise Operator Restrictions

Rule	Explanation
Only works with integers	Can't use with float, double
Used on byte, short, int, long, char	
Boolean version exists too (&&, `	

✅ Use Cases

- Performance optimization
- Cryptography (XOR is very common)
- Low-level hardware programming
- Permission/flag systems (e.g., file permissions)
- Compression or graphics programming

🔚 Summary Table

Operator	Use When...	Bit Rule
&	You want bits **ON in both**	1 & 1 = 1
`	`	You want bits **ON in any**
^	You want **different bits** to be 1	1 ^ 0 = 1, 1 ^ 1 = 0
~	You want to **invert bits**	~0101 = 1010
<<	You want to multiply by 2^n	x << 1 = x * 2
>>	You want to divide by 2^n	x >> 1 = x / 2
>>>	You want logical (unsigned) shift	No sign propagation

Would you like a real-world problem using bitwise flags or permissions to practice?

🔍 What is Operator Precedence and Associativity in Java?

When an expression has **multiple operators**, Java needs a **rule** to decide **which operator to evaluate first**. That's where **precedence** and **associativity** come in.

✅ 1. Operator Precedence

- **Precedence** means the **priority level** of an operator.
- Operators with **higher precedence** are **evaluated first**.

🧠 Example:

```
int result = 10 + 2 * 5;
```

👉 Here, * has higher precedence than +, so:

2 * 5 = 10 → then 10 + 10 = 20

✅ 2. Associativity

- If two operators have **same precedence**, associativity decides the **evaluation direction**:
  - **Left to Right** (most common)
  - **Right to Left** (e.g., assignment =, unary)

🧠 Example:

```
int x = 5;
x *= 2 + 3;
```

👉 Here, + has higher precedence, so it runs first:

2 + 3 = 5 → then x *= 5 → x = 25

⚠️ Common Precedence (High to Low)

1. () – Parentheses
2. ++, --, +, - (Unary)
3. *, /, %
4. +, -
5. ==, !=, >, <
6. &&, ||
7. =, +=, -=

Want a visual cheat sheet or real coding examples for this? Let me know!

Question :

Is it **BODMAS** rule ?

Great question!

❌ **No, Java does not strictly follow the BODMAS rule.**

BODMAS is a **math rule**:

Brackets, **O**rders (powers, roots), **D**ivision, **M**ultiplication, **A**ddition, **S**ubtraction

But in **Java**, we follow:

✅ **Operator Precedence & Associativity**

Java has its **own precedence rules**, which are **similar** to BODMAS in basic arithmetic, but it also includes:

- **Logical operators** (&&, ||)
- **Bitwise operators** (&, |, ^)
- **Assignment operators** (=, +=, etc.)
- **Unary and ternary operators**

🔍 **Example:**

```
int result = 10 + 2 * 3;
```

📌 Java evaluates * before +, just like BODMAS.
✅ So, yes — for basic math operators (+, -, *, /), **Java behaves like BODMAS**.

But Java expressions go **beyond math**, so it needs its **own full precedence table**.

Difference between expressions and statements in Java:

◆ Expression

An **expression** is a piece of code that **produces a value**.

✓ *Examples:*

```
5 + 3          // value: 8
a * b          // value: result of multiplication
x++            // value: original x, then increment
"Hello" + name // value: string concatenation
```

- **Always returns a result**
- Can be used **inside statements**

◆ Statement

A **statement** is a complete **instruction** to be executed by the Java compiler/interpreter.

✓ *Examples:*

```
int x = 5;          // declaration + assignment
System.out.println(x); // method call
x++;                // increment statement
if (x > 5) { ... }  // control statement
```

- May **contain expressions**
- Ends with a **semicolon (;)**
- Does **not always return a value**

🔁 In short:

- ☞ **Expression**: produces a value
- ☞ **Statement**: performs an action

 Think of it this way:

"**Expressions** make data.
Statements do things."

Want a real code example that highlights both in action?

Here's a simple **Java code example** that clearly shows the **difference between expressions and statements**:

✅ **Example:**

```
public class ExpressionVsStatement {
    public static void main(String[] args) {

        // 🔹 Expression
        int a = 10;          // '10' is an expression → value
        int b = 5 + 2;       // '5 + 2' is an expression → value 7

        // 🔹 Statement
        int sum = a + b;     // Entire line is a statement
                             // 'a + b' is the expression inside it

        // 🔹 Expression used in a print statement
        System.out.println("Total: " + sum); // Statement
                                             // '"Total: " + sum' is expression

        // 🔹 Control statement with expression
        if (sum > 15) {              // 'sum > 15' is an expression → returns true/false
            System.out.println("High"); // Statement
        }
    }
}
```

🔍 **Breakdown:**

Line	Expression(s)	Statement
`int a = 10;`	`10`	✅ Yes (assignment)
`int b = 5 + 2;`	`5 + 2`	✅ Yes (assignment)
`int sum = a + b;`	`a + b`	✅ Yes
`System.out.println("Total: " + sum);`	`"Total: " + sum`	✅ Yes
`if (sum > 15) { ... }`	`sum > 15`	✅ Yes (control flow)

Question :

Expression Evaluation with Real Code Examples.

Let's break down how Java evaluates expressions step by step:

```
public class ExpressionDemo {
    public static void main(String[] args) {
        int a = 10, b = 5, c = 2;

        // Example 1: Arithmetic Expression
        int result1 = a + b * c;
        // Output: 10 + (5 * 2) = 20
        System.out.println("Result1: " + result1);

        // Example 2: Relational + Logical
        boolean result2 = (a > b) && (b < c);
```

```
        // Output: (10 > 5) && (5 < 2) → true && false → false
        System.out.println("Result2: " + result2);

        // Example 3: Assignment with Expression
        int result3 = 0;
        result3 += a + b;
        // result3 = result3 + (10 + 5) → 0 + 15 = 15
        System.out.println("Result3: " + result3);

        // Example 4: Pre-increment vs Post-increment
        int x = 5;
        int y = ++x + x++;
        // Step-by-step: ++x → 6, then x++ → use 6 then increment to 7 → y = 6 + 6 = 12
        System.out.println("x: " + x + ", y: " + y);
    }
}
```

💡 **Pro Tip:** Use System.out.println() after each line when learning — it helps you *see* how Java processes the logic.

Basic Question :

[**Note:** While learning this Java topic, I faced the following questions and doubts. They might not be directly related to the current topic.]

If I write System.out.println("Add: " + a + b);, it concatenates. But if I write System.out.println("Add: " + (a + b));, it adds. Why does this happen?

Great question!

Yes, you're absolutely right — and here's **why** it behaves that way:

Java's + Operator Has Two Meanings:

1. **Arithmetic addition** → if both operands are numbers
2. **String concatenation** → if at least one operand is a String

---

❌ Example:

```
int a = 10;
int b = 5;

System.out.println("Add: " + a + b);
```

What happens:

- "Add: " + a → "Add: 10" (String)
- "Add: 10" + b → "Add: 105" (Still String)
- ✅ **Output**: Add: 105 → **Wrong addition, it's concatenation**

✅ Correct Way:

```
System.out.println("Add: " + (a + b));
```

What happens:

- (a + b) → 15 (number addition first)
- "Add: " + 15 → "Add: 15" (concatenated at the end)
- ✅ **Output**: Add: 15 → **Correct result**

Rule to Remember:

➡ **Parentheses () force arithmetic before String concatenation.**
➡ Without them, Java evaluates left to right and treats everything after a String as **String.**

Java Type Promotion Rules (Very Important!)

Absolutely! Here's a **complete list of all possible binary combinations** of arithmetic operations between Java **primitive numeric data types**:

✅ Java Numeric Data Types:

byte, short, char, int, long, float, double

🔁 Java Type Promotion Reminder:

🔹 **All operands smaller than int (i.e., byte, short, char) are promoted to int** before any operation.

- Example: byte + byte → both promoted to int → result is int
- Example: char + short → both promoted to int → result is int

🔹 **If one operand is larger (wider), the smaller type is promoted to the larger type.**

- Example: int + long → int promoted to long → result is long
- Example: float + int → int promoted to float → result is float

Order of promotion:

byte → short → int → long → float → double
(char → int)

📋 All Possible Combinations and Result Types

Operand 1	Operand 2	Result Type
byte	byte	int
byte	short	int
byte	char	int
byte	int	int
byte	long	long
byte	float	float
byte	double	double
short	short	int
short	char	int
short	int	int
short	long	long
short	float	float

Operand 1	Operand 2	Result Type
short	double	double
char	char	int
char	int	int
char	long	long
char	float	float
char	double	double
int	int	int
int	long	long
int	float	float
int	double	double
long	long	long
long	float	float
long	double	double
float	float	float
float	double	double
double	double	double

🧩 Sample Code: All Combinations

```
public class AllCombinations {
    public static void main(String[] args) {
        byte b = 10;
        short s = 20;
        char c = 'A'; // ASCII 65
        int i = 30;
        long l = 40L;
        float f = 50.5f;
        double d = 60.5;

        // byte combinations
        System.out.println("byte + byte = " + (b + b));
        System.out.println("byte + short = " + (b + s));
        System.out.println("byte + char = " + (b + c));
        System.out.println("byte + int = " + (b + i));
        System.out.println("byte + long = " + (b + l));
        System.out.println("byte + float = " + (b + f));
        System.out.println("byte + double = " + (b + d));

        // short combinations
        System.out.println("short + short = " + (s + s));
        System.out.println("short + char = " + (s + c));
        System.out.println("short + int = " + (s + i));
        System.out.println("short + long = " + (s + l));
        System.out.println("short + float = " + (s + f));
        System.out.println("short + double = " + (s + d));

        // char combinations
        System.out.println("char + char = " + (c + c));
        System.out.println("char + int = " + (c + i));
        System.out.println("char + long = " + (c + l));
        System.out.println("char + float = " + (c + f));
        System.out.println("char + double = " + (c + d));
```

```
        // int combinations
        System.out.println("int + int = " + (i + i));
        System.out.println("int + long = " + (i + l));
        System.out.println("int + float = " + (i + f));
        System.out.println("int + double = " + (i + d));

        // long combinations
        System.out.println("long + long = " + (l + l));
        System.out.println("long + float = " + (l + f));
        System.out.println("long + double = " + (l + d));

        // float combinations
        System.out.println("float + float = " + (f + f));
        System.out.println("float + double = " + (f + d));

        // double combinations
        System.out.println("double + double = " + (d + d));
    }
}
```

Output (Simplified):

```
byte + byte = 20
byte + short = 30
byte + char = 75
byte + int = 40
byte + long = 50
byte + float = 60.5
byte + double = 70.5
short + short = 40
short + char = 85
short + int = 50
short + long = 60
short + float = 70.5
short + double = 80.5
char + char = 130
char + int = 95
char + long = 105
char + float = 115.5
char + double = 125.5
int + int = 60
int + long = 70
int + float = 80.5
int + double = 90.5
long + long = 80
long + float = 90.5
long + double = 100.5
float + float = 101.0
float + double = 111.0
double + double = 121.0
```

Question :

I have a Student class and a main method. I want to pass values to the Student object using a constructor. Which class do I need to import into which class?

If you have a Student class with a constructor to initialize its fields, and a main method (usually in another class like Main or Test), **you don't need to import anything if both classes are in the same package**.

Case 1: Both classes in the same package (or default package)

- Just create a Student object in your main method by calling the constructor.
- No import needed.

```
// Student.java
public class Student {
    String name;
    int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

// Main.java (or in same file with main method)
public class Main {
    public static void main(String[] args) {
        Student s = new Student("Alice", 20);  // Passing values via constructor
        System.out.println("Student Name: " + s.name + ", Age: " + s.age);
    }
}
```

Case 2: Classes in different packages

- You **need to import** the Student class in the class that has the main method.

```
// package school;
package school;

public class Student {
    String name;
    int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
// package app;
package app;

// Import Student class from school package
import school.Student;
```

```
public class Main {
    public static void main(String[] args) {
        Student s = new Student("Bob", 22);
        System.out.println("Student Name: " + s.name + ", Age: " + s.age);
    }
}
```

Summary:

- If classes are **in the same package**, no import needed.
- If classes are in **different packages**, import the Student class in the class containing the main method.

This topic—**passing values to a class using constructors and importing classes**—comes under these core Java concepts:

1. **Classes and Objects**
    - Creating objects of a class using constructors.
    - Initializing object state via constructors.
2. **Constructors**
    - Special methods used to initialize new objects.
3. **Packages and Import Statements**
    - Organizing classes into packages.
    - Using import to access classes from other packages.
4. **Basic Java Program Structure**
    - How different classes interact in a program.

So, you can say it mainly falls under:

Object-Oriented Programming (OOP) fundamentals and **Java Packaging & Importing**.

If you want, I can help you organize these topics with examples and explanations!

Question :

Error : The method println(boolean) in the type PrintStream is not applicable for the arguments (void)

The error:

The method println(boolean) in the type PrintStream is not applicable for the arguments (void)

means you are trying to print the **result of a method that returns void**, but println expects a value to print.

Why this happens:

You are calling:

System.out.println(student.calculateTotalAndAverage());

but **calculateTotalAndAverage() returns void**, meaning it doesn't return any value.

println cannot print void, so the error occurs.

How to fix:

1. Check your Exp02_Student class's calculateTotalAndAverage() method.

- If it returns void, **don't use it inside println**.
- Instead, call the method alone, then print whatever values it calculates using other getters or the toString() method.

Example:

```
student.calculateTotalAndAverage();  // just call the method, don't print it

System.out.println(student.toString());  // print details including total & average
```

Summary:

Replace this line:

System.out.println(student.calculateTotalAndAverage());

with:

student.calculateTotalAndAverage();

If you want to print total and average inside toString(), make sure your toString() method returns a String containing those values.

If you want, share your Exp02_Student class code, and I can help you fix it exactly!

Date : 01-August-2025.